深度学习实验报告

2025-2026 学年第二学期

学院：	国豪书院
专业：	数据科学与大数据技术（精英班）
授课教师：	程大伟
学号：	2351569
姓名：	孙雨霏
日期：	2026.3.25

基于循环神经网络的唐诗自动生成

一、实验目的

本实验旨在利用循环神经网络（RNN）及其改进模型长短期记忆网络（LSTM），构建一个能够自动创作七言绝句的语言模型。通过本实验，预期达到以下具体目标：

1.1 深入理解序列建模原理

• 掌握递归机制：理解 RNN 如何通过隐藏层状态实现对文本时序信息的捕捉，使网络具备处理变长序列的记忆能力。

•攻克长程依赖难题：通过对比实验，深刻理解 LSTM 相比传统 RNN 在解决梯度消失、捕捉远距离语义关联（如诗歌首尾呼应）方面的技术优势。

1.2 实践端到端的 NLP 开发流程

• 数据工程能力训练：掌握从原始语料到张量数据的转换，包括字符清洗、词汇表构建及 Index 映射。

• 精准数据清洗实践：针对七言绝句的格律要求，实践通过特定长度筛选实现数据节奏对齐，验证“数据质量决定生成效果”的工程准则。

1.3 探究受限文本生成策略

•掌握概率采样技术：学习如何通过温度采样和重复惩罚平衡生成诗句的稳定性与多样性。

• 格律硬约束实现：探索在神经生成模型中引入人工干预逻辑，实现符合人类审美格律（七言、押韵）的创意文本生成。

1.4 优化模型在受限算力下的表现

• 收敛性能调优：在 Windows CPU 环境下，通过调整 Batch Size、学习率调度及训练轮数，寻求 Loss 下降与计算时间的最优平衡点。

• 过拟合与泛化的平衡：通过观察训练 Loss 曲线，掌握在小规模高质量数据集上实现适度过拟合以保证生成诗句通顺性的技巧。

二、模型原理解析

在自然语言处理任务中，文本具有显著的时间序列特征。本实验对比并应用了以下三种主流的循环神经网络结构：

2.1 RNN (Recurrent Neural Network)

传统全连接神经网络在处理序列数据时存在明显短板，其无法有效应对序列间的时间依赖关系。在诸如诗歌生成这类任务中，每个汉字的生成并非孤立，而是与前后字符紧密相连。而循环神经网络（RNN）的出现为解决这一问题提供了有效途径，它通过引入隐状态构建起独特的循环结构，将前一时刻的信息传递至当前时刻。如此一来，当网络处理当前汉字时，

能够“铭记”之前生成的字符信息，进而具备短期记忆能力，为处理序列数据奠定了基础。

从数学层面来看，RNN 的工作机制有着清晰的表达。在时刻 t，隐状态h_t的更新遵循特定公式，通常表示为$h_t=\sigma(W_h h_{t-1}+W_x x_t+b)$。其中，$h_{t-1}$是上一时刻的隐状态，$x_t$是当前时刻的输入，$W_h$、$W_x$为权重矩阵，b 是偏置项，σ是激活函数。该公式精准地描述了 RNN 如何结合历史信息与当前输入来更新隐状态，体现了其处理序列数据的内在逻辑。

然而，RNN 并非完美无缺，在反向传播过程中暴露出严重局限性。随着序列长度的增加，梯度需要跨越多个时间步进行连乘计算。这一过程极易引发梯度消失或梯度爆炸问题。梯度消失会使网络难以学习到长距离依赖关系，例如在诗歌生成中，诗头与诗尾的意境呼应这种长距离的语义关联就很难被捕捉到，导致标准 RNN 很难有效处理超过 10 个步长以上的序列信息，限制了其在复杂序列任务中的应用。

2.2 LSTM (Long Short-Term Memory)

LSTM 的结构设计独具匠心，旨在攻克标准 RNN 难以处理长距离依赖的难题。它引入了贯穿整个序列的“主干道”——细胞状态，为信息的长距离传递提供了稳定通道。同时，精心设计了遗忘门、输入门和输出门这三个“门”结构，对信息流转进行精密调控。遗忘门负责筛选，丢弃上一时刻细胞状态中无用的信息；输入门决定当前时刻哪些新信息能够存入细胞状态；输出门则依据当前细胞状态，确定输出何种隐状态。这种巧妙设计使 LSTM 能够有效捕捉长距离依赖关系。

在本实验的 rnn.py 里，RNN_model 类采用了 2 层 LSTM 堆叠结构（num_layers = 2）。这种多层结构并非简单的叠加，而是具有显著优势。它能够深入挖掘文本特征，从不同层次对唐诗进行分析和理解，进而学习到唐诗中跨度较长的语义规律以及复杂的韵律结构，为生成高质量的符合格律的唐诗提供有力支持。

2.3 GRU (Gated Recurrent Unit)

GRU 作为 LSTM 的一种精巧变体，在结构设计上进行了创新优化。它将 LSTM 原有的遗忘门、输入门和输出门简化为重置门与更新门这两个关键“门户”。重置门用于决定如何将新的输入信息与之前的记忆相结合，更新门则控制着有多少旧信息需要被保留以及多少新信息需要被加入。与此同时，GRU 大胆地将细胞状态与隐状态合二为一，极大地简化了网络结构，使得信息传递更加直接高效，减少了不必要的计算环节。

在优势对比方面，GRU 展现出了多方面的卓越特性。从参数量来看，GRU 的参数量相较于 LSTM 减少了约三分之一，这一特点在面对较小的数据集时优势尤为明显，能够有效降低过拟合的风险，使模型更具泛化能力。在训练效率上，由于结构更为简单，GRU 的训练速度通常比 LSTM 更快，能够节省大量的时间和计算资源。而在性能表现上，在众多自然语言处理任务中，GRU 不仅能够达到与 LSTM 相当的水平，甚至在某些情况下还能取得更优的效果，堪称是平衡性能与算力的绝佳选择。

三、诗歌生成过程叙述

本实验的诗歌生成流程是一个端到端的深度学习流水线，主要分为数据预处理、模型构建与训练、以及带硬干预逻辑的推理生成三个阶段：

3.1 数据预处理

为确保生成的诗歌严格遵循七言绝句的格律规范，本阶段对原始语料展开了全面且细致的清洗与结构化处理。这一系列操作旨在为模型训练提供高质量、标准化的数据输入，从而提升诗歌生成的准确性与文学性。

精准筛选环节是保障数据质量的关键步骤。通过调用 process_poems_seven_only 函数，从 poems.txt 文件中精准提取标准的七言绝句。筛选标准极为严格，要求全诗必须由 4 句构成，每句需包含 7 个汉字以及 1 个标点，总长度固定为 32 个字符。同时，对包含特殊符号（如括号、书名号等）的非规范诗句进行过滤，确保进入后续处理的数据均为符合七言绝句基本格式的诗句。

标记与分词操作进一步优化了数据结构。在每首诗的开头添加起始符 G (start_token)，结尾添加结束符 E (end_token)。这两个特殊标记的作用至关重要，它们能够帮助模型准确识别序列的边界，明确诗歌的起始与结束位置，从而更好地学习诗歌的结构特征，提高生成诗歌的连贯性与完整性。

词表构建是数据处理的重要环节。通过统计所有字符的出现频次，并按照频次进行降序排列，建立起字符到数字索引的映射表 word_int_map。这一映射表为后续的模型训练提供了基础，使得模型能够将字符转换为数字形式进行处理，同时也方便对字符进行统一管理和分析。

张量化与批处理操作则将文本数据转换为适合模型训练的格式。通过 PoemDataset 类，将文本数据转换为长整型张量，这种数据格式能够高效地被模型处理。同时，利用 DataLoader 实现数据的随机打乱与按批次读取。随机打乱数据可以避免模型在训练过程中因数据顺序而产生偏差，按批次读取数据则能够提高训练效率，充分利用计算资源，加速模型的收敛过程。

3.2 模型构建与训练

在模型架构设计上，采用了深度循环神经网络，其核心目标是深度学习汉字之间错综复杂的序列依赖关系，从而精准把握诗歌中字词间的内在逻辑与语义脉络，为生成高质量的诗歌奠定基础。

词嵌入层是模型处理输入数据的第一步。它运用 nn.Embedding 将高维稀疏的字符索引巧妙地映射为 128 维的连续稠密向量。这种转换不仅降低了数据的维度，还使字符具有了更丰富的语义表示。随后接入的 LayerNorm 技术，能够加速模型在训练过程中的收敛速度，同时有效提高训练的稳定性，确保模型能够更高效地学习数据特征。

核心编码器部分选用了两层堆叠的 LSTM 结构，隐藏层维度设定为 256。LSTM 凭借其独特的门控机制，包括遗忘门、输入门和输出门，在处理长序列数据时展现出显著优势。它

能够有效缓解长序列训练中容易出现的梯度消失问题，从而深入捕捉诗句之间长距离的语义关联，使模型能够理解诗歌中前后句之间的深层含义和逻辑关系。

```
# LSTM层 - 使用2层，batch_first=True便于处理数据
self.lstm = nn.LSTM(
    input_size=embedding_dim,
    hidden_size=lstm_hidden_dim,
    num_layers=2,
    batch_first=True,
    dropout=0.2
)
```

在模型的训练环节，损失计算与优化策略起着关键作用。模型输出经过 F.log_softmax 处理后得到概率分布，损失函数采用 nn.NLLLoss，并特别设置 ignore_index = 0 以忽略填充字符对损失计算的影响，确保损失计算的准确性。优化策略方面，使用 Adam 优化器，并引入 ReduceLROnPlateau 学习率调度器。当检测到平均损失在多个 Epoch 内不再下降时，该调度器会自动调小学习率，帮助模型跳出局部最优解，进一步寻找全局最优解，从而提升模型的性能和生成诗歌的质量。

3.3 推理生成逻辑

为有效弥补纯概率模型在生成格律诗过程中存在的局限性，本实验在推理阶段创新性地引入了“硬干预”逻辑，从多个维度对诗歌生成过程进行精准把控，以提升生成诗歌的质量和格律符合度。

引导词机制为用户参与诗歌生成提供了入口。用户可指定 begin_word 作为首个生成的字符，模型会以[G, begin_word]的输入序列为起点，开始递归预测后续字符。这一机制使得生成的诗歌能够从用户指定的字符展开，增加了诗歌生成的灵活性和个性化，满足了用户对于诗歌起始内容的特定需求。

节奏硬干预确保了诗歌的物理节奏严格对齐格律要求。在循环生成过程中，设置了字符计数器 word_count。当计数达到 7 时，系统会强行屏蔽所有汉字的生成概率，强制输出逗号（第 1、3 句后）或句号（第 2、4 句后）。通过这种精确的控制，使得诗歌在语句停顿和句末标点使用上符合七言绝句的节奏规范，增强了诗歌的韵律感和节奏感。

```
# 精准匹配：七言绝句通常包含 4 句，每句 7 字 + 1 个标点 = 8 字符
# 总长度应该是 8 * 4 = 32 字符
if len(content) == 32 and content[7] in '，。' and content[15] in '，。':
    content = start_token + content + end_token
    poems.append(content)
```

押韵约束是提升诗歌文学性的关键环节。利用 pypinyin 库获取汉字韵母，模型会记录第二句末尾字符的韵母。在第四句最后一个汉字生成时，通过 sample_with_rhyme 函数对不押韵的候选字施加极大的概率惩罚。这种强制性的押韵约束，使得生成的诗歌在韵律上更加和谐优美，符合传统诗歌的审美标准，提高了诗歌的艺术价值。

```
    # 如果是第二句末尾（sentence_count为1时），记录韵脚
    if sentence_count == 1:
        last_char = vocabularies[generated_ids[-1]]
        first_rhyme = get_rhyme_group(last_char)

    sentence_count += 1
    word_count = 0
else:
    # 未满 7 字，屏蔽标点和结束符
    logits[comma_id] = -1e9
    logits[period_id] = -1e9
    logits[word_int_map.get(end_token, 0)] = -1e9

    # 如果是第四句末尾，尝试匹配第二句韵脚
    current_target_rhyme = first_rhyme if (sentence_count == 3 and word_count == 6) else None

    if current_target_rhyme:
        word_id = sample_with_rhyme(logits, current_target_rhyme, vocabularies, temperature)
    else:
        word_id = sample_with_repetition_penalty(logits, generated_ids, penalty=1.5, temperature=temperature)

    word_count += 1
```

多样性与重复惩罚机制则从内容丰富度和避免重复的角度对诗歌生成进行优化。温度控制通过调整 Softmax 的温度系数，在生成结果的确定性与随机性之间找到平衡。温度系数较高时，生成结果更具随机性，多样性增强；温度系数较低时，生成结果更倾向于确定性，更符合已有模式。重复惩罚机制针对已在当前诗中出现过的字符，降低其在下一时刻被选中的概率，有效避免了循环神经网络常见的“词汇死循环”现象，使生成的诗歌词汇更加丰富多样，内容更加生动有趣。

```
# 对已经出现过的字符降低其 logits 值
for char_id in set(generated_ids):
    if char_id < len(logits):
        if logits[char_id] > 0:
            logits[char_id] /= penalty
        else:
            logits[char_id] *= penalty

logits = logits / temperature
```

四、实验结果展示

4.1 训练过程记录

在本实验精心搭建的训练流程中，模型历经了 100 个 Epoch 的持续迭代。随着训练的逐步推进，损失函数清晰地呈现出明显的下降趋势。这一积极变化直观地反映出模型在不断学习与优化过程中，正逐渐深入地掌握七言绝句独特的用词规律以及严谨的结构特征，为生成高质量的七言绝句奠定了坚实基础。

训练初期，当 Epoch 1 顺利完成时，平均损失 Avg Loss 高达 6.3610。此时的模型宛如初涉知识领域的新手，处于刚刚起步的阶段。由于尚未充分学习到七言绝句的相关特征，在预测字符时具有较大的随机性，生成的字符组合往往缺乏逻辑性和连贯性，难以形成有意义

的诗句。

```
============================================================
Epoch   1 完成 | Avg Loss: 6.3610
============================================================

============================================================
Epoch   2 完成 | Avg Loss: 5.8803
============================================================

============================================================
Epoch   3 完成 | Avg Loss: 5.4711
============================================================

============================================================
Epoch   4 完成 | Avg Loss: 5.2015
============================================================

============================================================
Epoch   5 完成 | Avg Loss: 4.9967
============================================================
```

随着训练的深入，进入训练中期，Epoch 50 完成时，平均损失 Avg Loss 降至 3.0350。此时的模型已经取得了显著的进步，开始能够生成具有一定意义的词组。不仅如此，它还初步掌握了七言绝句 7 字一句的节奏特点，生成的诗句在形式上逐渐向标准的七言绝句靠拢。

```
============================================================
Epoch  50 完成 | Avg Loss: 3.0350
============================================================
```

而当训练进入后期，Epoch 100 完成时，平均损失 Avg Loss 稳定在 2.6643。损失值趋于稳定，这表明模型已经基本收敛。此时生成的诗句在语义连贯性上有了极大提升，诗句之间过渡自然，而且在意境营造方面也有了显著进步，能够展现出一定的文学韵味和艺术美感。

```
============================================================
Epoch 100 完成 | Avg Loss: 2.6643
============================================================
```

4.2 诗歌生成结果

实验采用硬干预逻辑（强制标点、自动押韵及重复惩罚），以指定词汇作为开头生成的七言绝句结果如下：

```
============================================================
唐诗生成器 - 生成阶段（最终版）
============================================================

  - 匹配到的七言绝句数量: 7260
main.py:288: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module i
mplicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pyt
orch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`.
This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless th
ey are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use
 case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  rnn_model.load_state_dict(torch.load('./poem_generator_rnn', map_location='cpu'))
【日】日日河边多别离，月明流落在人间。今朝独向松间听，只见桃花一片天。
  - 匹配到的七言绝句数量: 7260
【红】红粉青青万里深，满筵帘下啭春声。琼枝尽日无人到，自有江边一夜风。
  - 匹配到的七言绝句数量: 7260
【山】山头水落人间半，重见河桥瘴涨州。为问吴江三百里，谁家华发向高楼。
  - 匹配到的七言绝句数量: 7260
【夜】夜无香火弄愁茶，客国江南杨柳春。日暮玉轮花霁后，笑看双蝶亦难寻。
  - 匹配到的七言绝句数量: 7260
【湖】湖上荒台不见人，风姿清习咏秋风。南山落日花先晚，一夜春泉一寸棱。
  - 匹配到的七言绝句数量: 7260
【海】海上高楼望远峰，洛阳晴路复何朝。重来马上浑红黛，不觉风吹狼藉桡。
  - 匹配到的七言绝句数量: 7260
【月】月落星稀天欲明，野人争利出乡回。汉家自有山僧地，不独行人是断追。
```

4.3 多样性演示

为了验证模型的生成灵活性，实验针对同一开头词“春”进行了多次采样生成，结果显示模型在相同输入下能产出意境截然不同的诗句：

```
============================================================
多样性演示 - 同一开头生成多首诗
============================================================

  - 匹配到的七言绝句数量: 7260
【春】(1) 春来曾住此中看，鸟散无人洗眼看。欲认与君歌一曲，莫教黄菊醉淹函。
  - 匹配到的七言绝句数量: 7260
【春】(2) 春江春水漱莓苔，一半沙崩万洞中。赢丽不堪怜客处，朝朝双户入云同。
  - 匹配到的七言绝句数量: 7260
【春】(3) 春前朝露有花开，闲步东流人不知。月暮寒松过云雨，还家更有听声迟。
```

五、实验总结

通过本次基于循环神经网络（RNN）的唐诗自动生成实验，从模型训练到多首七言绝句的成功生成，得出以下结论与体会：

5.1 模型表现与形式控制

在风格捕捉方面，实验结果令人瞩目。基于 LSTM 的多层循环神经网络展现出了强大的学习能力，它深入剖析唐诗的文本特征，成功学习到了唐诗独特的用词风格。不仅如此，对于唐诗中基本的意向组合，模型也能精准把握，使得生成的诗歌在词汇运用和意象搭配上，都带有浓郁的唐诗韵味，为还原唐诗的文学气质迈出了坚实一步。

格律实现是本次实验的关键突破。在推理阶段，通过巧妙引入“硬干预”逻辑，模型有效克服了纯概率生成所带来的随机性问题。在字符生成过程中，模型严格遵循每句 7 字的要求，精准自动切换标点，在第一、三句后使用逗号，第二、四句后使用句号。更为出色的是，模型还能确保第四句末尾自动押韵，使得生成的诗歌在韵律上和谐优美，完全符合七言绝句的格律规范，为诗歌的节奏感和音乐性提供了有力保障。

从结构模拟的角度来看，生成的诗歌成果斐然。在视觉形式上，这些诗歌已经与真实的七言绝句极为接近，无论是诗句的排列方式，还是整体的布局结构，都达到了较高的相似度。这一成果充分展示了深度学习在处理具有严格格律要求的文学体裁时的巨大潜力，为未来利用人工智能进行文学创作和研究开辟了新的道路，让我们看到了科技与文学融合的无限可能。

5.2 采样策略的关键作用

在实验推进过程中，一个棘手的问题逐渐浮现：当采用贪心搜索策略，即单纯选择概率最大的字符来生成文本时，生成的文本极易陷入词汇重复的死循环。例如，模型可能会反复生成“看、看、看”这样单调且无意义的字符组合，这不仅严重影响了生成文本的质量，还使得文本缺乏应有的丰富性和逻辑性，难以满足诗歌创作对于多样性和美感的要求。

为了有效解决这一问题，实验引入了带温度控制的采样和重复惩罚机制。温度控制机制如同一个智能调节器，通过对采样过程的干预，影响字符被选择的概率分布；而重复惩罚机制则像一个严格的监督者，对已经出现过的词汇施加一定的惩罚，显著降低其再次被选中的概率。在这两种机制的协同作用下，生成的诗词灵活性得到了极大提升，用词也更加丰富多样，避免了因词汇重复而导致的单调和乏味。

此外，温度系数的调整在生成过程中起着至关重要的平衡作用。通过灵活改变温度系数，我们能够在生成的“确定性”与“随机性”之间找到一个恰到好处的平衡点。较高的温度系数会增加生成的随机性，使诗句富有新意和创意，展现出独特的意境；而较低的温度系数则会提

高生成的确定性，让诗句更加符合逻辑和常规。这种平衡使得模型能够产出意境各异、风格多样的诗句，为诗歌创作带来了更多的可能性和惊喜。

5.3 局限性与改进方向

当前模型虽在形式上满足了七言绝句的基本要求，然而在语义层面却暴露出明显不足。由于采用字符级生成方式，模型对汉语词组的整体把握能力欠佳。部分生成的诗句尽管表面通顺，但在深层逻辑的串联以及文学意境的营造上，缺乏统一性和深度，难以呈现出唐诗那种精妙绝伦的韵味和内涵，这成为制约模型生成高质量诗歌的关键因素。

针对这一问题，未来可从两方面着手改进。一方面，引入预训练模型，基于更高维度的词向量或者对预训练大型语言模型（如 BERT、GPT 系列）进行微调。借助这些模型强大的语言理解和生成能力，为诗歌赋予更强的逻辑连贯性和情感深度，使诗句在表意和抒情上更加自然流畅、富有感染力。另一方面，尝试将古诗词的意向知识图谱融入模型，利用其中丰富的文化知识和意象关联，指导模型在特定主题下生成更具文化底蕴和内涵的内容，提升诗歌的文学价值。